안녕하세요.
Jquery 강좌중인 Falcon 입니다.

오늘 알려드릴 예제는 노드의 생성, 변경, 삭제에 관한 내용 입니다. 여러분들이 웹을 만들 때에 필요한 기술 중 하나이지만 javascript가 아닌 jquery로 만들게 되실 겁니다. 그렇다고해서 역시나 javascript를 모르시면 안된다고 저는 생각합니다. 그럼 먼저 추가를 하는 방법에 대해서 설명해드리도록 하겠습니다.

appendChild(child)	노드의 마지막 자식으로 주어진 엘리먼트 추가
insertBefore(new Element, refernceElement)	2번째 인자 앞에 추가

이 부분에 대해서는 글로 알려드리는 것 보다 실행을 해보면서 알려드리는게 더 쉬우니 실행화면을 보면서 확인해 봅시다. 그리고 위의 메소드를 제외하고도 2개의 메소드가 더 필요하게 됩니다.

document.createElement(tagname)	엘리먼트 노드를 추가
document.createTextNode(data)	텍스트 노드를 추가

추가에 관해서는 일단 이 4개의 메소드가 필요합니다. 지금부터 여러분들에게 이 부분을 알려드리도록 하겠습니다.

```
8   <body>
9     <ul id="target">
10      <li>html</li>
11      <li>css</li>
12    </ul>
13
14    <input type="button" onclick="callAppendChild()" value="append()">
15    <input type="button" onclick="callinsertBefore()" value="insert()">
16  </body>
```

index.html

여기서 여러분들에게 설명드릴 부분은 없지만 onclick는 한번 클릭 할 때 마다 실행 됩니다. 이 부분은 추후 이벤트에 대해 배우면서 여러분들에게 다시 소개를 하도록 하겠습니다.

```javascript
function callAppendChild() {
  var target = document.getElementById('target');
  var li = document.createElement('li');
  var text = document.createTextNode('javascript');

  li.appendChild(text);
  target.appendChild(li);
}

function callinsertBefore() {
  var target = document.getElementById('target');
  var li = document.createElement('li');
  var text = document.createTextNode('jquery');

  li.appendChild(text);
  target.insertBefore(li, target.firstChild.nextSibling.nextSibling);
}
```

master.js

이게 오늘의 소스코드 입니다. 뭔가 엄청 복잡해진 듯한 느낌인데 일단 appendChild를 하는 방법부터 알아봅시다.

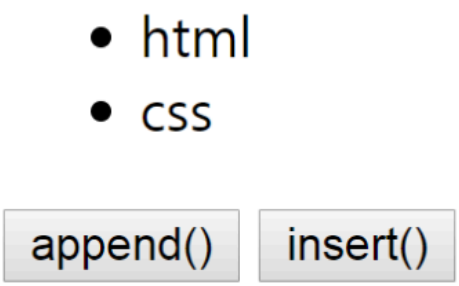
```javascript
1  function callAppendChild() {
2    var target = document.getElementById('target');
3    var li = document.createElement('li');
4    var text = document.createTextNode('javascript');
5
6    li.appendChild(text);
7    target.appendChild(li);
8  }
```

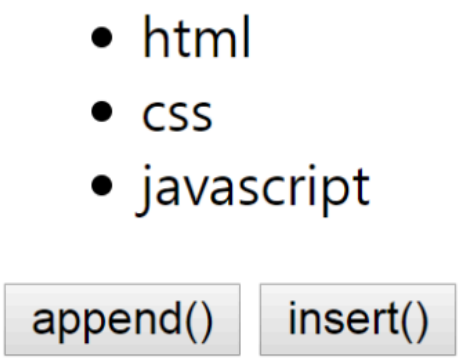
master.js

위는 master.js의 일부분입니다. 2행은 제어대상 찾기라는 것은 알고 계실테고 3행에서 보이는 createElement는 쉽게 말하면 li라는 태그를 추가하겠다라고 보시면 됩니다. 즉, <li> </li> 태그가 생성 된 것 입니다. 4행은 쉽게 말하면 TEXT_NODE라고 생각하시면 됩니다. <li>태그와 </li>태그 안에 들어갈 text를 입력했다고 보시면 됩니다. 여기서 중요한 것은 3행과 4행은 아직 결합이 안되어 있는 상태라는 것 입니다. 이를 결합하기 위해서는 6행의 코드가 필요합니다. li.appendChild는 노드의 마지막 자식인데 li태그의 마지막 자식은 #text입니다. 그렇기 때문에 text를 추가해주게 되면 그 때 완성된 li 태그가 나타나게 됩니다.

<li>javascript</li>

드디어 틀이 완성되었으니 적용을 해주어야 합니다. 7행 target.appendChild는 target 자식의 마지막 노드로 추가를 하겠다 무엇을 우리가 6행에서 만들어 놓은 li태그를 즉 이를 실행하면 이렇게 됩니다.





html과 css밖에 없었는데 javascript라는 li태그가 생성 된 것을 볼 수 있습니다.

```
Elements  Console  Sources  Network  »
<!doctype html>
<html lang="en" dir="ltr">
▶<head>…</head>
▼<body>
  ▼<ul id="target">
      <li>html</li>
      <li>css</li>
...   <li>javascript</li> == $0
    </ul>
    <input type="button" onclick="callAppendChild()" value="append()
    ">
    <input type="button" onclick="callinsertBefore()" value="insert()
    ">
  </body>
</html>
```

그리고 Elements에서도 정상적으로 추가되는 것을 볼 수 있습니다. 다음으로는 insertBefore 입니다.

```
10  function callinsertBefore() {
11    var target = document.getElementById('target');
12    var li = document.createElement('li');
13    var text = document.createTextNode('jquery');
14
15    li.appendChild(text);
16    target.insertBefore(li, target.firstChild.nextSibling.nextSibling);
17  }
```

mster.js(일부분)

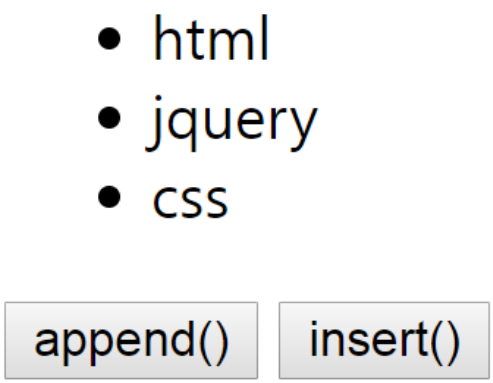
코드를 보면 딱히 달라진 부분은 없습니다. 15행까지는 append와 동일합니다. 다만 insertBefore는 여러분들이 원하는 위치에 태그를 추가 할 수 있습니다. 다만 2번째 인자값을 통해서 제가 fistChild와 nextSibling에 대해서는 여러분들이 원하는 위치로 이동할 줄 알아야한다고 했었습니다. 그게 바로 여기서 이게 쓰이기 때문이였습니다. 물론 이렇게 fistChild와 nextSibling를 통해서 여러분들에게 알려드리고 있지만 사실 이보다 아주 조금 더 간단한 방법이 있습니다. 한 번 보여드리도록 하겠습니다.

```
function callinsertBefore() {
  var target = document.getElementById('target');
  var li = document.createElement('li');
  var text = document.createTextNode('jquery');

  li.appendChild(text);
  target.insertBefore(li, target.firstElementChild.nextSibling);
}
```

master.js(일부분)

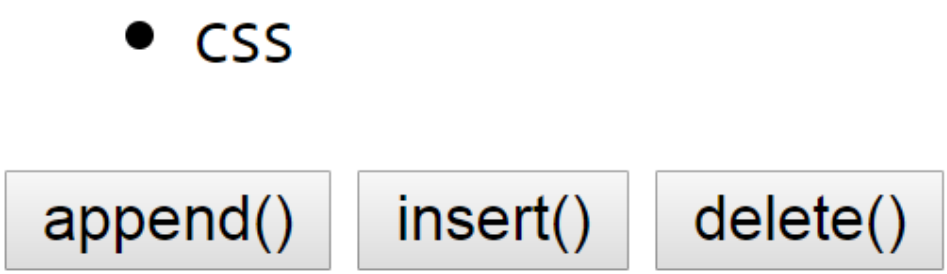
코드가 조금 길어지지만 firstElementChild를 이용하면 위에선 3개를 써야할 것은 하나 줄일 수 있습니다.
이러한 방법외에도 이런저런 방법들이 더 있을거라고 생각합니다.




```
function calldelete() {
  var target = document.getElementById('target');
  target.removeChild(target.firstElementChild);
}
```

master.js

index.html 에서는 버튼을 하나 추가하셔야 합니다. 제일 아래에 코드는 배포해드리겠지만 직접 추가도 해보는 연습을 해보시기 바랍니다. 그래서 index.html을 여기서는 제외시켰습니다. 저는 target이 ul태그에 있기 때문에 ul태그 아래 첫번째 자식을 삭제해라라는 것으로 만들었습니다. 이를 쉽게하기 위해서는 id값을 사용하면 좀 더 쉽고 빠르게 할 수 있을 것 같습니다. 여러분들도 여러 시도를 해보면서 실력을 늘리시기 바랍니다.



이렇게 저는 delete를 추가시켰습니다. 그리고 이 버튼을 누르면 첫번째 자식을 삭제가 되는 것을 볼 수 있습니다.
원래 기본은 html과 css가 나오게 됩니다. 그리고 마지막으로 변경하는 방법을 알려드리도록 하겠습니다.

```
1  function callReplaceChild() {
2    var a = document.createElement('a');
3    a.href = '#';
4    a.appendChild(document.createTextNode('css web'));
5
6    var target = document.getElementById('target');
7    target.replaceChild(a, target.firstElementChild.nextElementSibling);
8  }
```

master.js

3행 부분을 보면 a.href가 보입니다. 이는 속성을 의미한다는 것을 알 수 있습니다. 여러분들이 추가할 태그에 속성을 지정하는 것입니다. 저는 아무곳도 이동하지않는 '#'을 붙여주었습니다. 4행에서는 조합을 하는데 ()괄호안에 있는 인자 값을 이렇게도 할 수 있다라는 것을 여러분들에게 보여드리고 있습니다. 7행 부분을 보면 첫번째 인자값은 위에서 만들어 놓은 'a'태그 입니다. 두번째 인자값은 위치인데 이 위치를 보면

- html
  [css web]

append() insert() Replace() delete()

<li>태그가 사라지게 됩니다. 원래 제가하려던 목적은 <li>태그안에 이를 추가하는 <a>태그를 집어 넣으려고 했던 것인데 아쉽게도 실패했습니다. 실패의 원인은 부모와 자식 관계에서만 접근이 가능하다고 합니다.

```
9   <ul id="target">
10    <li>html</li>
11    <li>css</li>
12  </ul>
```

<ul>태그 입장에서 자식 태그는 #text와 <li>태그 입니다. 그런데 제가 원하는 css의 위치의 부모는 <li>태그 입니다. 즉, <ul>태그에서는 <li>태그안에 #text인 css에 접근을 할 수가 없었습니다. <ul>태그에 있어서는 <li>태그 안의 #text는 부모가 아니기 때문에 이는 불가능 했습니다. 그래서 코드를 수정했습니다.

```
1  function callReplaceChild() {
2    var a = document.createElement('a');
3    a.href = '#';
4    a.appendChild(document.createTextNode('css web'));
5
6    var target = document.getElementById('target');
7    var li = target.getElementsByTagName('li');
8    li[1].replaceChild(a, li[1].firstChild);
9  }
```

master.js

7행을 추가하고 8행을 수정함으로서 부모와 자식관계가 되면서 이를 수정할 수 있었습니다.

- html
- [css web]

append() insert() Replace() delete()

위의 수정사항으로 제가 원하는대로 출력을 할 수 있었습니다. 생활코딩에서는 id 값 target을 왜 <li>태그 안에 넣었는지 생각을 했었는데 위치를 찾는 연습 도중에 이를 발견해버렸습니다. 이를 발견함으로서 여러분들에게 알려드릴 것은 ReplaceChild(), removeChild(), insertBefore() 에서도 마찬가지로 부모,자식을 지켜주셔야 한다는 것을 알려드립니다. 이점을 주의하시고 사용하시면 될 거 같습니다.

index.txt

master.txt

배우면서 작업한 코드 입니다. 위의 코드가 잘 보이지 않거나 읽으면서 해결을 못하신 분들은 이 코드를 참고해서 배우시면 될 거 같습니다. 그럼 저는 이만~!

이것으로 [Jquery 강좌] 제 23강 노드의 변경 API 편을
마치도록 하겠습니다.